

基于 Spring Cloud 实现业务系统微服务化的设计与实现

文/王方旭

摘要

微服务架构是一种先进的架构模式,它将业务系统彻底组件化、服务化的思想是 IT 应用应对互联网思路、云端部署的最佳解决方案之一,而 Spring Cloud 对微服务架构提供了全面的技术支持,是实践微服务架构案例最优技术框架。基于 Spring Cloud 框架实现微服务架构的应用显著地提升了响应速度和持续集成能力,实现了开发、测试、部署运维的一体化。

【关键词】微服务 Spring Cloud Spring Boot

1 引言

互联网、云计算技术的高速发展,人们对信息化系统服务的依赖程度日益加深。随着服务、数据的爆炸式增长,自上世纪 90 年代以单体架构建设的 IT 服务系统越来越无法满足现实要求。虽然 SOA(面向服务的架构)的出现一定程度上能够缓解单体架构的弊端,但 SOA 架构系统的服务通信机制的高成本和对企业服务总线的依赖决定了仍不是最佳解决方案。“微服务”概念适时的出现为我们解决了问题,微服务架构的一切皆服务的理念和在云计算领域的优异表现获得了普遍认可。

本文介绍了一种基于 Spring Cloud 框架实现业务系统微服务化的案例,结合微服务架构思想和 Spring Cloud 框架,将以单体架构建设的应用系统改造成的典型的微服务框架系统,改造后,系统显著提升了应用开发、测试、部署运维一体化的能力。

2 微服务

微服务的概念是 2014 年 3 月由 Martin Fowler 在他的文章《Microservices》中首次提出,作为一种先进的架构模式,其核心思想就是:应用是由多个小的、相互独立的、可采用分布式部署的服务组成,服务的开发、部署在各自独立的进程中,不同服务通过一些轻量级交互机制来通信,例如 RPC、HTTP 等,服务可独立扩展伸缩,每个服务定义了明确的边界,不同的服务可以采用不同的编程语言来实现,

由独立的团队来维护。目前,随着互联网思维和云平台部署越来越趋势化,微服务架构在应用系统的建设中所体现出的价值越来越大。

微服务架构具有如下优势:

(1) 微服务架构将业务系统彻底的组件化、服务化,微服务专注于业务逻辑,服务功能简单,边界清晰,复杂度低,接口明确,服务利于开发、部署。

(2) 服务耦合度低,每个服务是一个微型的应用,有完整的架构,可独立部署。

(3) 微服务架构允许根据服务的功能和团队的自身条件选择不同的技术路线,允许服务由不同的开发团队开发。

(4) 优良的容错机制和熔断机制,保障微服务之间交互的友好性。

微服务架构虽是一种先进的架构模式,但仍有它的不足:

(1) 微服务架构的应用是分布式系统,相较于单体架构的应用系统具有较大的复杂性,尤其是需要在 RPC 或者消息传递之间选择并完成进程间的通讯机制;此外,消息传递、服务调用的性能相对于单体应用会有所下降。

(2) 分布式系统带来的数据库数据的一致性对开发者提出更大的挑战。

3 Spring Cloud

在 Spring 生态圈内基于 Spring Boot 实现的 Spring Cloud 框架越来越受到业界的关注, Spring Cloud 是一系列框架、组件的有序集合,拥有功能完善的、轻量级的微服务实现组件,例如服务发现治理、服务容错、服务网关、服务配置、负载均衡、消息总线、服务跟踪等方面均有经过实践检验的成熟组件。

如图 1,展示了基于 Spring Cloud 各组件的完整架构图。

(1) Eureka 负责微服务架构中服务治理功能,实现服务实例的自动化注册与发现。

(2) Zuul 在客户端与服务端配置实现,作为微服务网关,可有效的减少客户端与服务端的交互次数,易于监控,易于认证,动态路由。

(3) Ribbon 是负载均衡器,负责自动从 Eureka Server 发现服务,然后基于某种负载均衡算法帮助服务消费者去请求服务。

(4) HyStrix 是熔断器,为避免发生雪崩效应而引入,对服务延迟和故障提供更加强大容错能力。

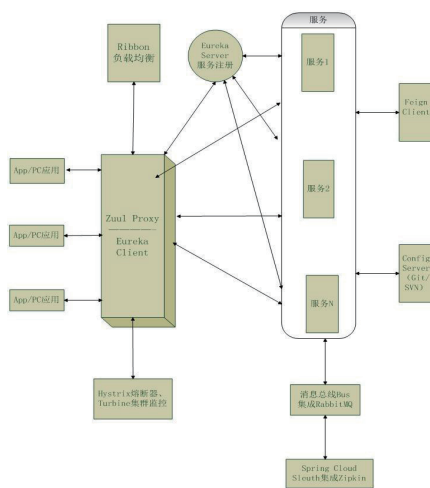


图 1

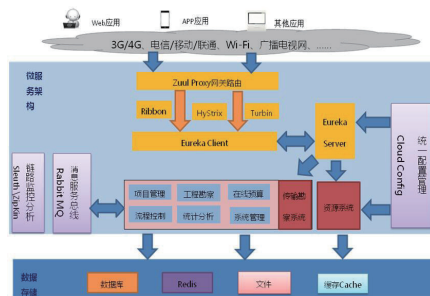


图 2

(5) Turbine 是聚合 HyStrix 监控数据的工具,结合 HyStrix 监控微服务集群。

(6) Config Server 为分布式系统提供统一管理的配置,在 Spring Cloud 框架中使用有利于配置的集中管理,动态调整配置,且自动更新配置。

(7) Spring Cloud Bus 管理和传播服务之间的消息,通过集成当前最主流的消息中间件 RabbitMQ 可有效的在各分布式服务之间解耦,提供高易用性、高扩展性的通信功能。

(8) Spring Cloud Sleuth 集成 Zipkin,为应用实现分布式的链路监控分析方案。

综上,当客户端请求服务时,Zuul 网关将请求动态转发,Ribbon 会根据服务网关的配置实现负载均衡,Eureka 获知请求的服务,在服务注册中心获得服务;微服务向客户端提供服务时,会读取在 Config Server 配置的统一配置;而 HyStrix 和 Turbine 负责监控服务的熔断情况,并给予图形化展示;RabbitMQ 在各服务之间通信; Sleuth+Zipkin 追踪每个请求,

<< 下转 61 页

基于 MIPS 架构的软件性能探究工具

文/梁成豪

摘要

如今,人们更多的是利用高级语言(如C语言)去开发软件。而随着业务和计算的复杂度提高,以及软件运行平台本身的影响,如何理想地探究单个软件性能成了一个难以完成的任务。本文就在计算机体系结构仿真工具 SimpleScalar 之上,通过引入统计方式方法,来使在纯净环境下探究 MIPS 架构下 C 应用程序的性能成为可能。

【关键词】软件性能 计算机体系结构 SimpleScalar MIPS

1 引言

如今,高级语言(如C语言)已经成为大量被应用于系统、应用软件的开发。软件的性能好坏在对其进行大规模应用时在设计极差

的地方可以被宏观的观察出来,但是也很难定位到问题所在,即便是可以定位到问题所在,也很难分析得很细微以达到对软件相应设计较差的代码进行改进的目的。如今有一些软件性能分析工具如 linux 的 perf,以及C语言库头文件 <time.h> 中也有类似的函数和相应数据结构。但是 perf 无法做到精确统计应用程序每行代码是否有 cache miss 等有害性能地方也很难精确计算执行时间,文章[1]中在龙芯 2F 中设计的 perf 软硬件,也只是更贴近理论值。因为除了该应用程序,计算机中总是有其它程序来运行,可以近乎完全认定它们对该应用程序运行产生了影响,例如它们的运行污染了 cache,甚至加将此应用程序的内容从 cache 中踢出;而对于 <time.h> 中关于时钟周期数的统计,它们仍然是内嵌于程序内的,毫无疑问会污染整个程序运行的运行。此外,对于一些常量的限定,比如 CLOCKS_PER_SEC,对于时间计算更是一个误差所在。因为它们忽略了

诸如 DVFS 之类的硬件层面对 CPU 状态的改变。

实际上,实际系统中很难有完全精确的工具来对应用程序的性能进行不间断的精确追踪。这样的精确追踪。而计算机体系结构仿真器是一个很好的解决方案。

计算机体系结构仿真器主要是用来在系统层面探究和优化计算机体系结构参数设计的工具。通常而言,它们详细建模了某一计算机体系结构内部部件,诸如 CPU 执行部件、cache 等。软件通过编译可以在仿真工具上进行运行,运行之后仿真器给出运行结果统计,从而验证软件在某一组体系结构参数(通常包括 cache 相关性、内存读写端口数等)的性能。本文利用 SimpleScalar (SS) 为基础来开发 MIPS 架构软件性能探究工具。

2 工具及可执行文件格式简介

2.1 SimpleScalar 简介

<< 上接 60 页

记录请求的服务、耗时、网络延迟等数据以备后续优化系统时分析使用。

4 基于Spring Cloud的微服务架构案例

4.1 系统现状

传输勘察系统和资源系统是应用于通信行业有线传输的业务系统,提供传输勘察设计的全过程的业务管理。随着业务的发展,系统内功能的优化调整、系统间功能的调用和数据交流非常普遍的,传输勘察系统和资源系统必须进行有效地资源整合。系统是作为依附于业务的发展而建设的系统,由不同时间阶段、不同开发团队开发完成,并且传输勘察系统和资源系统采用了不同的技术路线。为了解决以上问题并使应用系统在将来业务拓展时有良好的应对能力,我们采用了微服务架构的思想作为改造思路,而 Spring Cloud 框架作为在云部署、虚拟化方面均有优异输出的分布式应用系统框架是实现微服务架构的最佳解决方案。

4.2 业务系统微服务框架

基于传输勘察系统和资源系统的现状和将来的发展趋势,我们设计了如图 2 的系统框架。

将各功能模块封装成微服务并部署在独立的服务器中,各微服务提供服务接口注册在 Eureka Server 服务注册中心中;微服务之间消息的管理和传播由 Rabbit MQ 完成,并部署负责链路监控分析的服务监控系统运行情况;对系统的访问由基于 Zuul Proxy 的网关进行转发。

5 总结

通过采用 Spring Cloud 框架对业务系统的改造,将业务功能复杂、耦合度高、系统维护成本大的系统改造成了独立部署、耦合度低、业务功能组件化的分布式系统,为系统的运营维护节省了极大的成本。随着业务的发展,无论是新功能的增减还是业务逻辑的优化,业务系统都能够做到快速响应,敏捷开发,灵活配置,快速部署,独立运行,为业务的发展提供了快速有力的支持。

参考文献

- [1]Fowler M, Lewis J. Microservices. <https://martinfowler.com/articles/microservices.html>. [2014-03-25].
- [2]王磊. 微服务架构与实践 [M]. 北京: 电子工业出版社, 2015.
- [3]周立. Spring Cloud 与 Docker 微服务架构实战 [M]. 北京: 电子工业出版社, 2017 (05).

作者简介

王方旭(1984-),男,山东省济宁市人。硕士学历。高级信息系统项目管理师,主要从事软件架构、微服务、分布式应用方面的研究与设计。

作者单位

中国移动通信集团设计院有限公司山东分公司
山东省济南市 250101